

	SR UNIVERSITY Campus Warangal
	Program: II - B. Tech (CS& AI) Batch: 66
	Name: Pranjal Kumar HT: 2503a51663
	Department: Computer Science and AI Semester: I
	AI Assisted Coding-Assignment:12 Date:08-10-2026

Q1 – AI-Assisted Bubble Sort Analysis

Problem / Starter Code:

```
numbers = [64, 34, 25, 12, 22, 11, 90]
```

```
# TODO:
```

```
# Implement Bubble Sort in ascending order.
```

```
# Count comparisons and swaps.
```

```
# Display the sorted list.
```

Student Tasks:

- Implement Bubble Sort.
- Use AI to explain one complete pass of Bubble Sort.
- Count comparisons and swaps.
- Test at least three input lists.
- Ask AI about an early-exit optimisation.
- Compare the original and improved versions.

```
ai assist coding > 5.py > ...
1  def bubble_sort(numbers):
2      arr = numbers[:] # work on a copy so the original list remains unchanged
3      comparisons = 0
4      swaps = 0
5      n = len(arr)
6
7      for i in range(n):
8          swapped = False
9
10         for j in range(0, n - i - 1):
11             comparisons += 1
12
13             if arr[j] > arr[j + 1]:
14                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
15                 swaps += 1
16                 swapped = True
17
18         print(f"Pass {i + 1}: array={arr}, comparisons={comparisons}, swaps={swaps}")
19
20         if not swapped:
21             break
22
23     return arr, comparisons, swaps
24
25
26 if __name__ == "__main__":
27     numbers = [64, 34, 25, 12, 22, 11, 90]
28     sorted_numbers, total_comparisons, total_swaps = bubble_sort(numbers)
29
30     print(f"Final sorted list: {sorted_numbers}")
31     print(f"Total comparisons: {total_comparisons}")
32     print(f"Total swaps: {total_swaps}")
33
```

Problems Output Debug Console Terminal Ports

```
● PS C:\Users\kpran\Desktop\2nd Year> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ;
● (.venv) PS C:\Users\kpran\Desktop\2nd Year> & "c:\Users\kpran\Desktop\2nd Year\ner-landslide-platform\.v
Pass 1: array=[34, 25, 12, 22, 11, 64, 90], comparisons=6, swaps=5
Pass 2: array=[25, 12, 22, 11, 34, 64, 90], comparisons=11, swaps=9
Pass 3: array=[12, 22, 11, 25, 34, 64, 90], comparisons=15, swaps=12
Pass 4: array=[12, 11, 22, 25, 34, 64, 90], comparisons=18, swaps=13
Pass 5: array=[11, 12, 22, 25, 34, 64, 90], comparisons=20, swaps=14
Pass 6: array=[11, 12, 22, 25, 34, 64, 90], comparisons=21, swaps=14
Final sorted list: [11, 12, 22, 25, 34, 64, 90]
Total comparisons: 21
Total swaps: 14
○ (.venv) PS C:\Users\kpran\Desktop\2nd Year> |
```

Output: Sorted = [11, 12, 22, 25, 34, 64, 90]; Comparisons = 21; Swaps = 14.

One complete pass:

64>34 → swap → [34,64,25,12,22,11,90]

64>25 → swap → [34,25,64,12,22,11,90]

64>12 → swap → [34,25,12,64,22,11,90]

64>22 → swap → [34,25,12,22,64,11,90]

64 > 11 → swap → [34, 25, 12, 22, 11, 64, 90]

64 < 90 → no swap → [34, 25, 12, 22, 11, 64, 90]. The largest element 90 is now at the end.

Three test cases:

[1, 2, 3, 4, 5] → [1, 2, 3, 4, 5], 4 comparisons, 0 swaps.

[5, 4, 3, 2, 1] → [1, 2, 3, 4, 5], 10 comparisons, 10 swaps.

The early-exit optimisation stops when a complete pass makes no swaps. This reduces unnecessary work for already sorted or nearly sorted data.

Complexity: average/worst case $O(n^2)$; best case with early exit $O(n)$; extra algorithmic space $O(1)$.

Q2 – Linear Search vs Binary Search

Problem / Starter Code:

numbers = [5, 12, 18, 23, 31, 45, 56, 72, 89]

target = 45

TODO:

Implement Linear Search.

Implement Binary Search.

Compare the number of comparisons.

Student Tasks:

- Implement Linear Search and Binary Search.
- Use AI to explain why Binary Search requires sorted data.
- Count comparisons for both algorithms.
- Test first, middle, last and absent elements.
- Compare the results and explain suitable use cases.
- Use AI to suggest a valid optimisation.

```
ai assist coding > 6.py > ...
1 def linear_search(values, target):
2     """Return (index, comparisons), or (-1, comparisons) if target is absent."""
3     comparisons = 0
4
5     for index, value in enumerate(values):
6         comparisons += 1
7         if value == target:
8             return index, comparisons
9
10    return -1, comparisons
11
12
13 def binary_search(values, target):
14     """Search a sorted list and return (index, comparisons)."""
15     low = 0
16     high = len(values) - 1
17     comparisons = 0
18
19     while low <= high:
20         middle = low + (high - low) // 2
21
22         comparisons += 1
23         if values[middle] == target:
24             return middle, comparisons
25
26         comparisons += 1
27         if values[middle] < target:
28             low = middle + 1
29         else:
30             high = middle - 1
31
32    return -1, comparisons
33
```

Problems Output Debug Console Terminal Ports

```
(.venv) PS C:\Users\kpran\Desktop\2nd Year> & "c:\Users\kpran\Desktop\2nd Year\ner-landslide-platform\.venv\Scripts\python.exe" "c:/Users/kpran/Desktop/2nd Year/ai assist coding/5.py"
(.venv) PS C:\Users\kpran\Desktop\2nd Year> & "c:\Users\kpran\Desktop\2nd Year\ner-landslide-platform\.venv\Scripts\python.exe" "c:/Users/kpran/Desktop/2nd Year/ai assist coding/6.py"
Linear Search and Binary Search comparison counts
First element (5): Linear Search -> index 0, 1 comparisons; Binary Search -> index 0, 5 comparisons
Middle element (31): Linear Search -> index 4, 5 comparisons; Binary Search -> index 4, 1 comparisons
Last element (89): Linear Search -> index 8, 9 comparisons; Binary Search -> index 8, 7 comparisons
provided target (45): Linear Search -> index 5, 6 comparisons; Binary Search -> index 5, 5 comparisons
absent element (100): Linear Search -> index -1, 9 comparisons; Binary Search -> index -1, 8 comparisons

Binary Search requires sorted data because it compares the target with the middle value and discards the half that cannot contain the target. Without sorted order, that half cannot safely be discarded.
Use Linear Search for small or unsorted lists, or when only one search is needed. Use Binary Search for large, sorted lists, especially when searching repeatedly.
Optimization: If many searches are needed, sort the data once and reuse Binary Search for each lookup; this avoids sorting again for every query.
```

For target 45: Linear Search finds it at index 5 after 6 comparisons. Binary Search finds it at index 5 after 3 comparisons.

Required tests:

First element 5 → Linear: 1 comparison; Binary: 3 comparisons; Found.

Middle element 31 → Linear: 5 comparisons; Binary: 1 comparison; Found.

Last element 89 → Linear: 9 comparisons; Binary: 4 comparisons; Found.

Absent element 50 → Linear: 9 comparisons; Binary: 3 comparisons; Not found.

Why Binary Search needs sorted data:

Binary Search compares the target with the middle element and discards one half of the list. This is safe only when the values are ordered; otherwise the discarded half may contain the target.

Optimisation: use Binary Search when the data is already sorted and there are many searches. Linear Search remains suitable for small or unsorted data.

Complexity: Linear Search $O(n)$ time and $O(1)$ extra space. Iterative Binary Search $O(\log n)$ time and $O(1)$ extra space.

Q3 – Compare Sorting Algorithms

Problem / Starter Code:

data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]

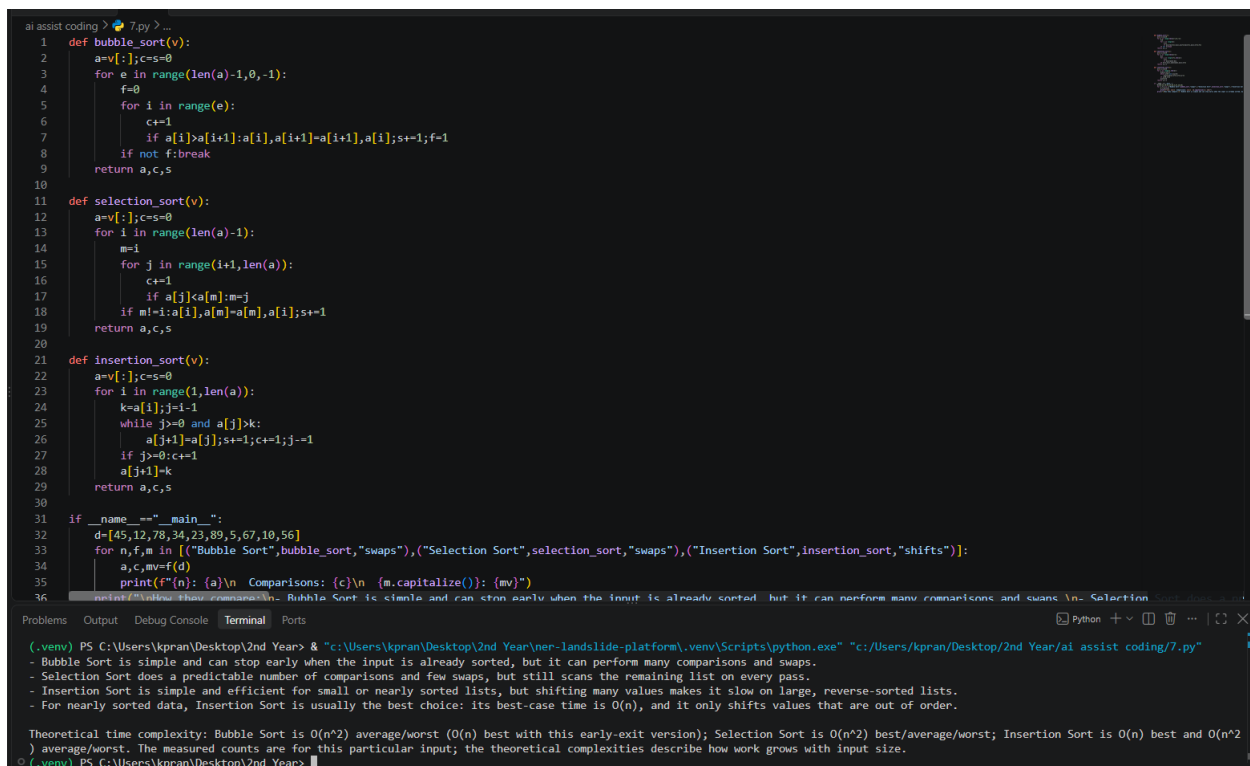
TODO:

Sort using Bubble Sort, Selection Sort and Insertion Sort.

Compare their performance.

Student Tasks:

- Implement all three sorting algorithms.
- Use the same input for each algorithm.
- Count comparisons and swaps/shifts.
- Use AI to explain strengths and weaknesses.
- Ask AI which algorithm suits nearly sorted data.
- Compare experimental results with theoretical complexity.



```
1 def bubble_sort(v):
2     a=v[:];c=s=0
3     for e in range(len(a)-1,0,-1):
4         f=0
5         for i in range(e):
6             c+=1
7             if a[i]>a[i+1]:a[i],a[i+1]=a[i+1],a[i];s+=1;f=1
8             if not f:break
9     return a,c,s
10
11 def selection_sort(v):
12     a=v[:];c=s=0
13     for i in range(len(a)-1):
14         m=i
15         for j in range(i+1,len(a)):
16             c+=1
17             if a[j]<a[m]:m=j
18             if m!=i:a[i],a[m]=a[m],a[i];s+=1
19     return a,c,s
20
21 def insertion_sort(v):
22     a=v[:];c=s=0
23     for i in range(1,len(a)):
24         k=a[i];j=i-1
25         while j>=0 and a[j]>k:
26             a[j+1]=a[j];s+=1;c+=1;j-=1
27         if j>=0:c+=1
28         a[j+1]=k
29     return a,c,s
30
31 if __name__=="__main__":
32     d=[45,12,78,34,23,89,5,67,10,56]
33     for n,f,m in [("Bubble Sort",bubble_sort,"swaps"),("Selection Sort",selection_sort,"swaps"),("Insertion Sort",insertion_sort,"shifts")]:
34         a,c,mv=f(d)
35         print(f"{n}: {a}\n Comparisons: {c}\n {m.capitalize()}: {mv}")
36         print(f"AI: {f} is {m} and can stop early when the input is already sorted but it can perform many comparisons and swaps\n- Selection Sort is simple and can stop early when the input is already sorted, but it can perform many comparisons and swaps.
37         - Selection Sort does a predictable number of comparisons and few swaps, but still scans the remaining list on every pass.
38         - Insertion Sort is simple and efficient for small or nearly sorted lists, but shifting many values makes it slow on large, reverse-sorted lists.
39         - For nearly sorted data, Insertion Sort is usually the best choice: its best-case time is O(n), and it only shifts values that are out of order.
40         Theoretical time complexity: Bubble Sort is O(n^2) average/worst (O(n) best with this early-exit version); Selection Sort is O(n^2) best/average/worst; Insertion Sort is O(n) best and O(n^2) average/worst. The measured counts are for this particular input; the theoretical complexities describe how work grows with input size.
```

All three produce: [5, 10, 12, 23, 34, 45, 56, 67, 78, 89].

Experimental comparison: Bubble Sort = 44 comparisons, 24 swaps; Selection Sort = 45 comparisons, 6 swaps; Insertion Sort = 31 comparisons, 24 shifts.

Strengths and weaknesses:

Bubble Sort: simple, but normally $O(n^2)$ and may perform many swaps. Early exit helps nearly sorted data.

Selection Sort: performs relatively few swaps, but still makes $O(n^2)$ comparisons.

Insertion Sort: very suitable for small or nearly sorted data because only misplaced elements need to move.

Nearly sorted data: Insertion Sort is the best choice among these three.

Theoretical complexity: Optimised Bubble Sort best $O(n)$, average/worst $O(n^2)$; Selection Sort best/average/worst $O(n^2)$; Insertion Sort best $O(n)$, average/worst $O(n^2)$.

Q4 – Optimising a Student Search and Sort Program

Problem / Starter Code:

```
students = [  
    ("Arun", 82),  
    ("Bala", 65),  
    ("Chitra", 91),  
    ("Divya", 74),  
    ("Esha", 88)  
]
```

TODO:

Find the highest mark.

Sort students by mark.

Search for a student by name.

Improve the solution using AI.

Student Tasks:

- Write functions for highest mark, sorting and name search.
- Create a straightforward solution first.
- Use AI to identify unnecessary operations.
- Ask AI for a more efficient approach.
- Implement only suggestions you understand.

- Compare original and improved time complexity.

```

1 def highest_mark(s):
2     return None if not s else sorted(s, key=lambda x: x[1])[-1]
3
4 def sort_students_by_mark(s):
5     return sorted(s, key=lambda x: x[1])
6
7 def search_student_by_name(s, n):
8     for x in s:
9         if x[0].casefold() == n.casefold(): return x
10    return None
11
12 def highest_mark_optimized(s):
13     return None if not s else max(s, key=lambda x: x[1])
14
15 if __name__ == "__main__":
16     students=[("Arun",82),("Bala",65),("Chitra",91),("Divya",74),("Esha",88)]
17     name="Divya"
18     print(f"Straightforward highest mark (sort then take last): {highest_mark(students)}")
19     print(f"Improved highest mark (single pass): {highest_mark_optimized(students)}")
20     print(f"Students sorted by mark: {sort_students_by_mark(students)}")
21     print(f"Search for {name!r}: {search_student_by_name(students, name)}")
22     print("\nUnnecessary operation identified:")
23     print("Search all students just to find the highest mark does extra work. A single-pass maximum checks each student once.")
24     print("\nTime complexity:")
25     print("- Highest mark, straightforward (sort then take last): O(n log n).")
26     print("- Highest mark, improved (single pass): O(n).")
27     print("- Sorting by mark: O(n log n) using Python's sorted().")
28     print("- Name search (linear scan): O(n) worst case; stops early if found.")
29     print("For many name searches, build a name-to-student dictionary once: O(n) setup, then O(1) average lookup per search, using O(n) extra space.")

```

Problems Output Debug Console **Terminal** Ports

```

(.venv) PS C:\Users\kpran\Desktop\2nd Year> & "c:\Users\kpran\Desktop\2nd Year\ner-landslide-platform\.venv\Scripts\python.exe" "c:/Users/kpran/Desktop/2nd Year/ai a
Time complexity:
- Highest mark, straightforward (sort then take last): O(n log n).
- Highest mark, improved (single pass): O(n).
- Sorting by mark: O(n log n) using Python's sorted().
- Name search (linear scan): O(n) worst case; stops early if found.
For many name searches, build a name-to-student dictionary once: O(n) setup, then O(1) average lookup per search, using O(n) extra space.
(.venv) PS C:\Users\kpran\Desktop\2nd Year>

```

Output: Highest = Chitra, 91. Sorted by mark = Bala–65, Divya–74, Arun–82, Esha–88, Chitra–91. Chitra search = (Chitra, 91).

AI-suggested improvement:

If the program performs many name searches, scanning the complete list for every search repeats work. Build a name index once and reuse it.

Complexity comparison: highest mark remains $O(n)$; sorting remains $O(n \log n)$ using Python's sorted; original name search is $O(n)$ per search, while indexed name search is $O(1)$ average after $O(n)$ index construction.

The index uses $O(n)$ extra space. It is useful when many name searches are performed; for only one tiny search, building an index may not be necessary.

Q5 – Optimisation Challenge: Best Combination

Problem / Starter Code:

```

items = [

    ("Laptop", 60000),

```

```
("Phone", 30000),  
("Tablet", 20000),  
("Watch", 10000),  
("Headphones", 5000)  
]
```

```
budget = 50000
```

```
# TODO:
```

```
# Find a combination of items whose total price
```

```
# is as high as possible without exceeding the budget.
```

Student Tasks:

- Design a solution to maximise total value without exceeding the budget.
- Start with a simple approach and explain it.
- Use AI to compare Brute Force, Greedy and Dynamic Programming.
- Determine which approach guarantees an optimal solution for this formulation.
- Implement the selected solution.
- Test at least three budgets.
- Explain time and space complexity and possible optimisation for larger inputs.

```
ai assist coding > 9.py > ...
1 def highest_mark_straightforward(s):
2     return None if not s else sorted(s, key=lambda x: x[1])[-1]
3
4 def highest_mark_optimized(s):
5     return None if not s else max(s, key=lambda x: x[1])
6
7 def sort_students_by_mark(s):
8     return sorted(s, key=lambda x: x[1])
9
10 def search_student_by_name(s, n):
11     for x in s:
12         if x[0].casefold() == n.casefold(): return x
13     return None
14
15 if __name__ == "__main__":
16     students=[('Arun',82),('Bala',65),('Chitra',91),('Divya',74),('Esha',88)]
17     print("Straightforward highest-mark result:", highest_mark_straightforward(students))
18     print("Optimized highest-mark result:", highest_mark_optimized(students))
19     print("Students sorted by mark:", sort_students_by_mark(students))
20     print("Search for 'Divya':", search_student_by_name(students, "Divya"))
21     print("\nOptimization explanation:")
22     print("Sorting the entire list just to find its highest mark is unnecessary. The optimized function scans each student once with max().")
23     print("For repeated name searches, create a dictionary keyed by normalized names once. This takes O(n) setup and O(n) extra space, then each lookup is O(1) on average.")
24     print("\nTime complexity:")
25     print("- Highest mark, straightforward (sort then last): O(n log n)")
26     print("- Highest mark, optimized (single scan): O(n)")
27     print("- Sorting by mark: O(n log n)")
28     print("- Name search (linear scan): O(n) worst case")

Problems Output Debug Console Terminal Ports
(.venv) PS C:\Users\kpran\Desktop\2nd Year> & "c:\Users\kpran\Desktop\2nd Year\ner-landslide-platform\.venv\Scripts\python.exe" "c:/Users/kpran/Desktop/2nd Year/ai as
(.venv) PS C:\Users\kpran\Desktop\2nd Year> & "c:\Users\kpran\Desktop\2nd Year\ner-landslide-platform\.venv\Scripts\python.exe" "c:/Users/kpran/Desktop/2nd Year/ai as
Straightforward highest-mark result: ('Chitra', 91)
Optimized highest-mark result: ('Chitra', 91)
Students sorted by mark: [('Bala', 65), ('Divya', 74), ('Arun', 82), ('Esha', 88), ('Chitra', 91)]
Search for 'Divya': ('Divya', 74)

Optimization explanation:
Sorting the entire list just to find its highest mark is unnecessary. The optimized function scans each student once with max().
For repeated name searches, create a dictionary keyed by normalized names once. This takes O(n) setup and O(n) extra space, then each lookup is O(1) on average.

Time complexity:
- Highest mark, straightforward (sort then last): O(n log n)
- Highest mark, optimized (single scan): O(n)
- Sorting by mark: O(n log n)
- Name search (linear scan): O(n) worst case
(.venv) PS C:\Users\kpran\Desktop\2nd Year>
```

For budget ₹50,000, the best combination is Phone + Tablet = ₹50,000.

Greedy vs Dynamic Programming:

Greedy makes the best local choice at each step, but for the general 0/1 item-selection formulation it does not guarantee the global optimum.

0/1 Dynamic Programming guarantees the optimal answer for this formulation because it considers the best achievable total for smaller budgets while ensuring each item is used at most once.

Test results:

Budget ₹50,000 → Phone + Tablet → ₹50,000.

Budget ₹65,000 → Laptop + Headphones → ₹65,000.

Budget ₹100,000 → Laptop + Phone + Watch → ₹100,000.

Complexity: Brute Force checks all subsets, $O(2^n)$ time. 0/1 Dynamic Programming is $O(nB)$ time and $O(B)$ extra space for budget B. For larger inputs, DP can be expensive when B is very large; reducing the state space or using a problem-specific optimisation may help.

Final choice: Dynamic Programming is selected because the assignment asks for a method that guarantees the optimal combination for the stated 0/1 formulation.